

ويرجّع تحليل السعرات ، Gemini سيرفر بسيط يستقبل صورة الأكل من الموقع ، يرسلها لـ
المفتاح يبقى هنا في السيرفر فقط ولا يظهر أبداً في كود الموقع

```
const express = require('express');
const cors = require('cors');
require('dotenv').config();

const app = express();
app.use(cors()); // قعك فقط بدل السماح للجميع origin في الإنتاج: حدد
app.use(express.json({ limit: '10mb' })); // تاج حد أعلى من الافتراضي base64 الصور

const GEMINI_API_KEY = process.env.GEMINI_API_KEY;
const GEMINI_MODEL = process.env.GEMINI_MODEL || 'gemini-1.5-flash';
const GEMINI_IMAGE_MODEL = process.env.GEMINI_IMAGE_MODEL || 'gemini-2.5-flash-im

if (!GEMINI_API_KEY) {
  console.error('❌ .env في ملف GEMINI_API_KEY لم يتم ضبط');
  process.exit(1);
}

app.get('/health', (req, res) => res.json({ ok: true }));

app.post('/api/analyze-food', async (req, res) => {
  try {
    const { image, mimeType } = req.body;
    if (!image) return res.status(400).json({ error: 'إرسال صورة' });

    const prompt = `حلل هذه الصورة لوجبة طعام وقدر القيمة الغذائية`.
    فقط بدون أي نص إضافي وبدون علامات كود، بهذا الشكل بالضبط JSON أعد
    {"name": "الوجبة بالعربي", "calories": عدد, "protein": عدد, "carbs": عدد, "fat": عدد, "

    const geminiRes = await fetch(
      `https://generativelanguage.googleapis.com/v1beta/models/${GEMINI_MODEL}:ge
    {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        contents: [{
          parts: [
            { text: prompt },
            { inline_data: { mime_type: mimeType || 'image/jpeg', data: image }
          ]
        }
      ]},
      generationConfig: { temperature: 0.2 }
```

```

    })
  }
);

const data = await geminiRes.json();

if (!geminiRes.ok) {
  console.error('Gemini error:', data);
  return res.status(502).json({ error: 'فشل الاتصال بـ Gemini', details: data.
}

const text = data.candidates?.[0]?.content?.parts?.[0]?.text || '';
const clean = text.replace(/```json|```/g, '').trim();
const start = clean.indexOf('{');
const end = clean.lastIndexOf('}');

let parsed = null;
try { parsed = JSON.parse(clean.slice(start, end + 1)); } catch (e) { /* تحليل

if (!parsed) return res.status(502).json({ error: 'Gemini تعذر فهم رد', raw:

res.json(parsed);
} catch (err) {
  console.error(err);
  res.status(500).json({ error: 'خطأ داخلي في السيرفر' });
}
});

// للتعقيد الصحيحة لتمرين معين (واقعية 2D) توليد صورة توضيحية
// ثري دي تفاعلية حقيقية، بل صورة يولدها الذكاء الاصطناعي توضح وضعية الجسم الصحيحة
app.post('/api/generate-exercise-image', async (req, res) => {
  try {
    const { exerciseName } = req.body;
    if (!exerciseName) return res.status(400).json({ error: 'اسم التمرين مطلوب' })

    const prompt = `صورة تعليمية واقعية توضح الوضعية الصحيحة لتمرين "${exerciseName}"
    حيح من الجانب، إضاءة استوديو نظيفة، خلفية صالة بسيطة، بدون أي نص مكتوب على الصورة

const geminiRes = await fetch(
  `https://generativelanguage.googleapis.com/v1beta/models/${GEMINI_IMAGE_MOD
  {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      contents: [{ parts: [{ text: prompt }] }],
      generationConfig: { responseModalities: ['IMAGE'] }
    })
  })

```

```

    }
  );

  const data = await geminiRes.json();
  if (!geminiRes.ok) {
    console.error('Gemini image error:', data);
    return res.status(502).json({ error: 'فشل توليد الصورة', details: data.error });
  }

  const parts = data.candidates?.[0]?.content?.parts || [];
  const imagePart = parts.find(p => p.inlineData || p.inline_data);
  const inline = imagePart && (imagePart.inlineData || imagePart.inline_data);
  if (!inline) return res.status(502).json({ error: 'لم يتم إرجاع صورة من Gemini' });

  res.json({ image: inline.data, mimeType: inline.mimeType || inline.mime_type });
} catch (err) {
  console.error(err);
  res.status(500).json({ error: 'خطأ داخلي في السيرفر' });
}
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`🟢 السيرفر شغال على المنفذ ${PORT}`));

```